

Building AI Application Challenge 2026

Builder Progress Log & Submission Workbook

Days 2–8 (Environment → Build → Evaluate → Deploy → Submit)

Use this document as a living workbook. Update it daily with links to your repo, demos, screenshots, and decisions.

0) Participant & Project Metadata

Participant Name	Emlin Bino
Email	eb4002@hw.ac.uk
Timezone	GST (Dubai) Currently in: CAT(Central African Time) - Zambia
Start Date	
Path (select one)	● ☐ Model Building ● ☒ LLM/API Integration ● ☐ No-Code/Low-Code
Team Members (if any)	None
Mentor / Reviewer (if any)	None
Project Repo URL (GitHub)	https://github.com/bluejay-19/Vault
Demo / App URL (if available)	https://vault-finance.streamlit.app/

Quick links (keep updated):

- GitHub repo (public link)
<https://github.com/bluejay-19/Vault>
- Figma design prototype (public link):
- <https://www.figma.com/make/eapTVx4008mcBEdVPo3hef/Vault-Personal-Finance-Tracker?code-node-id=0-9&p=f&t=S0X1cKokoT1s06YU-0&fullscreen=1>
- Deployment link (Streamlit)
<https://vault-finance.streamlit.app/>
- Demo video link (Google Drive)
https://drive.google.com/drive/folders/18mMUqSB1_plkDyKzOr2WHizwc-hzGeZ?usp=sharing
- LinkedIn progress post URL
https://www.linkedin.com/feed/update/urn:li:activity:7476509167361351680?utm_source=share&utm_medium=member_desktop&rcm=ACoAAFatjWkBDpTUZ6Q9oPQg6ZlCx-RLLGzqua4
- Tweet/X progress post URL (if applicable) – N/A
- Skool progress post URL (if applicable)
<https://www.skool.com/decoding-data-science-6929/day-8-vault-is-submitted?p=0c4e55cd>
- DDS Private Community post URL (if applicable)
<https://academy.decodingdatascience.com/c/#/post/13731>

1) Day 1 Idea Summary (from your Day 1 template)

Fill this section by copying your Day 1 submission. Keep it crisp but specific.

Project Title	Vault
Concept Summary	Vault is a personal finance tracker powered by an AI persona called Frank. Unlike generic finance apps. Vault doesn't just show you where your money went – Frank tells you exactly what he thinks about it, in plain, unfiltered terms. The core experience: a user uploads their monthly spending spreadsheet (a simple Excel file with dates and category columns works fine), and Vault analyses the data, generates interactive visualisations, and Frank delivers a personalized roast based on the real numbers. Users can also chat directly with Frank to ask questions about their finances. Because users log in, their data persists, meaning Frank gets more insightful over time, comparing this month to last month, spotting recurring patterns, and tracking whether the user is actually improving.
Target Audience	Primary audience: University students and young adults (18-28) who struggle to track where their money goes and would prefer more helpful insights of how to better spend and save their money. Secondary audience: Anyone who has a simple monthly spreadsheet and wants actionable and honest feedback on their spending habits.
Key Features (top 3–5)	1. Authentication and Accounts 2. Excel/CSV Upload 3. Dashboard and visualisations 4. Frank – The AI Roast persona 5. Income, Budget & Savings Tracking 6. Frank's Monthly Summary 7. Spending History
Technical Approach (high-level)	Python – Core language for all logic Streamlit – Frontend UI and multipage app structure Pandas – Excel/CSV parsing, data analysis, pattern detection Plotly – Interactive charts – pie, bar, line graphs

	Groq API – Powers Frank’s AI persona and chat responses Supabase – User authentication + persistent data storage Streamlit Community Cloud – Free deployment for the final app
Expected Challenges	• Supabase authentication setup • Varied excel formats from users • Streamlit UI limitations • Frank’s Roasts feeling generic • Persistent user sessions • Data being retrieved and used from the database in the app • Running out of time to complete all features
Expected Outcome (by Day 8)	A fully deployed working finance web app with: • Working user authentication and persistent accounts • Excel uploads and real data analysis using Pandas • Interactive Plotly charts showing spending patterns • A named AI persona (Frank) that roasts user’s based on real data (their data) • Running savings total and showing financial progress over time
Additional Notes	• The raccoon mascot (Frank) should be consistent across all pages, a small avatar in the nav, a larger illustration on the dashboard roast section, and a chat avatar. • Provide a downloadable sample Excel template in the app so users know exactly what format to upload • Keep the core loop (upload → charts → roast) as the absolute priority. Chat and history are stretch goals • After the challenge, the dream version of Vault is rebuilt with Flask + React + Supabase for a proper portfolio centrepiece The name Vault is a play on 'receipts' (evidence/proof) and 'wrecked' - memorable, on-brand, and self-explanatory

	Chat with Frank feature was moved to v2 to keep the 8-day scope focused on a complete, polished core experience
--	---

2) Architecture Snapshot (living design)

Update this whenever your design changes.

One-line problem statement	Students and young adults overspend without realising it because tracking finances feels tedious and existing tools give no honest, motivating feedback.
Primary user story / flow	User logs in → uploads monthly spending file + enters income/budget/savings → views dashboard with charts and Frank's AI summary → checks history to track progress over time Reference figure 2 below for visual
Inputs	Monthly spending CSV/Excel file (Date, Description, Amount, Category), income (number), budget (number), net savings (number, can be negative)
Outputs	Interactive dashboard (summary cards, donut chart, line chart), Frank's written AI summary, history page with month-by-month cards
Key components (UI, backend, storage, etc.)	Streamlit (UI), Supabase (auth + database), Pandas (parsing/analysis), Plotly (charts), Groq API (Frank's AI persona)
Data flow diagram (link/image)	The diagram illustrates the data flow for the Vault app. It starts with a User (stick figure) who uploads a file to a Streamlit instance (yellow oval). This Streamlit instance sends the file to a Pandas instance (red oval). Pandas stores the parsed data in a Supabase database (green cylinder). The Supabase database pulls data from another Streamlit instance (yellow oval), which then sends a spending summary to the Groq API (purple oval). The Groq API returns Frank's roast to the second Streamlit instance, which then displays everything back to the User. Figure 1: Data Flow Diagram - Vault App
Non-functional requirements (latency, cost, privacy, uptime)	Cost: free tier only (Supabase, Groq, Streamlit Community Cloud). Uptime: best-effort, free tier hosting.

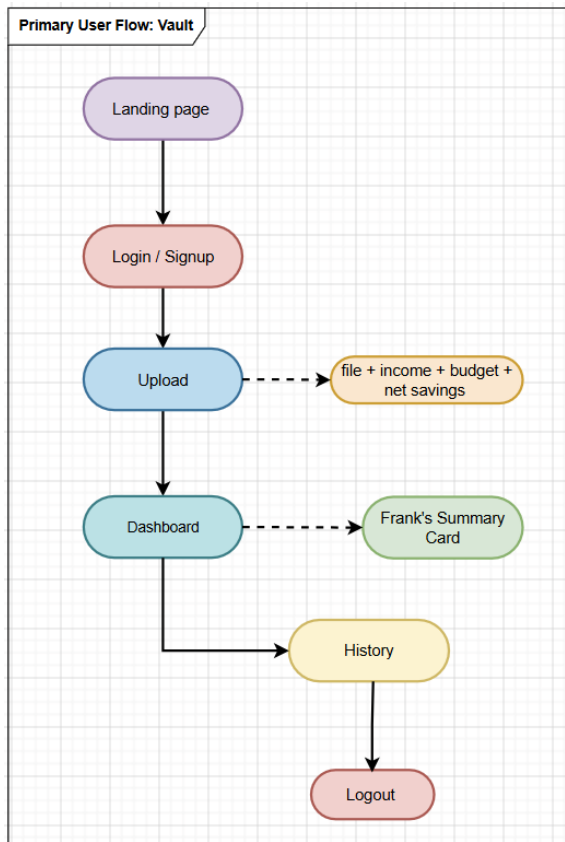


Figure 2: Primary User flow diagram

3) Data / Knowledge Sources Inventory

Day 2 focus: list your data sources clearly. This becomes critical for quality, evaluation, and judging.

Source	Type	Owner / License	Format	Size	Update freq	Access method	Preprocessing needed	PII/Sensitive?	Status
User-uploaded spending file	User-generated	Owned by user	CSV/XLSX	~1 month of transactions typically <500 rows	Monthly (one upload per month)	Streamlit file uploader	Parse columns, validate format, handle negative/missing values	Yes - financial data, private per user	In progress
User input fields (income, budget,	User-generated	Owned by user	Numeric form input	Single values per month	Monthly	Streamlit number_input	Validate non-negatives for income and budget	Yes - financial data	In progress

saving, month & year)									
Stored monthly records	Derived/processed	Owned by user, stored by app	Supabase table (Postgres)	Grows monthly per user	Per upload	Supabase python client	None-already structured on storage	Yes	Planned (Day 3)

If you are doing RAG / search:

- ☐ Corpus assembled (what content?)
- ☐ Chunking strategy defined (size/overlap)
- ☐ Embedding model selected
- ☐ Vector store selected (or file-based retrieval)
- ☐ Citation strategy (how will you show sources?)
- ☐ Data safety: remove secrets/PII and respect copyright

4) LLM / Model Selection & Experiments

Capture what you tried, what worked, and why you chose your final stack.

Candidate	Provider	Why considered	Prompting approach	Quality notes	Latency notes	Cost notes	Decision
Llama 3.370B	Groq	Used successfully in previous challenge (OctoPi), Fast inference and a generous free tier	Single system prompt defining Frank's persona + structured data summary as context	Strong personality – consistency, references specific numbers well when prompted clearly	Very fast – typically under 2 seconds	Free tier sufficient for this scale	✅ Selected
Gemini 2.0 Flash	Google	Free tier available, considered as an alternative option	System prompt + content	Good	Fast	Free However for proper use I think payment is required	Not selected due to quota errors on the free tier and version compatibility issues

Claude Sonnet API	Anthropic	Considered for better quality – idea to be used for future versions of the app that require additional features and need more power	Not Tested	N/A Would be considered one of the strongest models so far regards to analysing and providing feedback		Requires payment	Not selected, due to cost barrier for a student project, decided to stick with Groq mainly due to familiarity and will explore higher power models when developing future versions of the apps.
-------------------	-----------	---	------------	---	--	------------------	---

Prompt versions (for LLM/API or No-Code tools):

Prompt ID	Goal	System constraints	Few-shot examples?	Tools/Actions used	Notes / Results
frank_v1	Generates Frank's monthly spending summary	Must reference real numbers (amounts, categories), stay in character, end with a few actionable tips, include disclaimer it's not professional advice	No few-shot examples used – single structured system prompt	None – direct Groq API call with data summary as context	To be tested on Day 5

Model-building path only (if applicable):

Framework	e.g., PyTorch / TensorFlow / scikit-learn
Model family	e.g., classifier / regressor / transformer / CNN / etc.
Baseline approach	
Training data	
Validation strategy	
Key metrics	

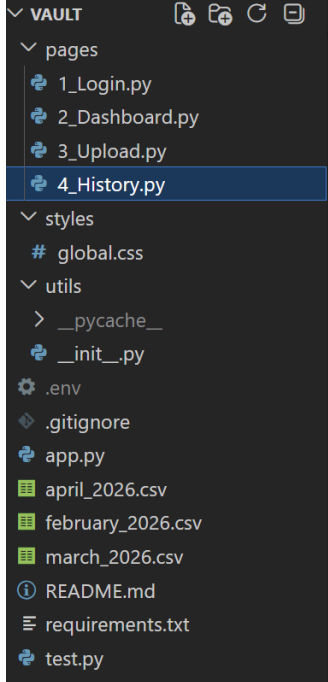
5) Day 2 — Environment Setup & Initial Development

Deliverable for Day 2: no full project submission today—prepare your environment + tech stack/tooling + initial work evidence.

Environment setup checklist (tick what applies):

- ☒ Repository created and initial commit pushed
- ☒ README created (project goal + how to run)
- ☒ .gitignore configured (no secrets)
- ☒ Python environment created (venv/conda) OR platform workspace created (no-code) – Using global Python not venv/conda
- ☒ Dependencies installed and pinned (requirements.txt)
- ☒ API keys stored safely (.env, secrets manager, or platform secrets)
- ☒ Basic 'hello world' run completed locally
- ☒ Basic API call tested (if using LLM/API) OR first workflow run (if no-code)
- ☒ Folder structure created (src/, data/, notebooks/, etc.)
- ☒ First prototype screen/flow created (UI stub acceptable)

Evidence (links):

GitHub repo URL	https://github.com/bluejay-19/Vault
Commit hash / tag	1a55055
Screenshot(s) link	 Figure 3: Project folder structure
Notes on setup decisions	Used Groq API Used Supabase for database to store persistent user account data Used Streamlit for frontend

Initial development log (what you built today):

- Project folder structure (pages/, styles/, utils/)
- requirements.txt with pinned dependency versions
- .gitignore configured to exclude .env
- Global dark theme CSS injected across all pages
- Landing page with hero section and feature cards
- Supabase client setup (get_supabase_client() helper)
- Login/Signup page with email + password auth
- Session stored in st.session_state on successful login

Blockers / issues encountered (and how you resolved them):

- Large HTML block in st.markdown() rendered as raw text instead of rendering — resolved by splitting one large markdown call into multiple smaller calls
- SupabaseException: supabase_key is required - caused by a typo in .env (SUPBASE_ANON_KEY missing a letter); fixed by correcting the variable name
- Missing openpyxl and plotly packages - installed via pip, not yet needed for Day 2 logic but added early since they're listed in requirements.txt

Day 3 — Building the Brain of Your App

Checklist:

- ☒ Core logic implemented (model/prompt/workflow)
- ☒ Data ingestion or API integration expanded beyond hello world
- ☒ 10–20 test questions/examples drafted (start your 'exam set')
- ☒ First measurable baseline created (even if rough)
- ☒ README updated with run instructions

Artifacts / notes:

What is the 'brain' of your app (1–2 sentences)?	The brain of Vault is a structured Groq prompt (llama-3.370b-versatie that takes real parsed spending data – total spent, budget, income, category breakdown; and generates a four-part roast: an opening observation, where spending is going wrong. What's working, and one specific actionable recommendation, all grounded in the user's actual numbers rather than generic advice.
Link to key code/workflow	pages/2_Dashboard.py – Groq prompt construction and API call (lines containing the prompt = f""" ... """ block and client.chat.completions.create() call) https://github.com/bluejay-19/Vault/blob/main/pages/2_Dashboard.py

Baseline results (short)	Tested 8 out of 20 documented scenarios across 6 real uploads on one test account. 2 bugs found: donut chart label overlap on very small category values, and misleading extreme percentage display on "vs Last Month" when consecutive months have been drastically different totals. Both bugs since fixed. All 8 tested scenarios passed on accuracy (correct biggest category, correct budget status, no crashes)
What you will improve next	Adding a direct "Ask Frank" interaction where users can ask a specific question about their spending and get a grounded answer, expanding test coverage to the remaining 12 scenarios, and reviewing prompt consistency on tone across repeated roast generations ("Another roast" button variety).

Decisions made today (why):

- Switched category set from the original lowercase 6 to an adaptive 8-category system, where missing categories default to 0 rather than rejecting the file chosen because real bank statements vary month to month and shouldn't require a fixed category list.
- uploads table in Supabase with 8 columns + foreign key to auth users
- Two RLS policies (SELECT & INSERT) so users only access their own data
- Month/year select box inputs (dynamic, pre-selects current month/year)
- Supabase insert on "Save to Vault" button with full try/except
- Also decided to drop the planned full chat page (Ask Frank) in favour of a single roast-card interaction for v1, prioritizing a polished, fully working core loop over a half-finished chat feature, given the 8-day timeline.
- Added a quick question feature below the spending summary given by Frank on the dashboard page that allows users to ask questions related to their finances
- Tested prompt injection resistance - Frank held character under a basic "ignore previous instructions: "attempt without dedicated guardrails, though a topic redirect instruction will still be added as a precaution
- Also tested multilingual input (French, Korean) – French queries handled fully in French with the correct context, Korean queries responded in English – which is an inconsistency to document rather than fix given the scope

Blockers / help needed:

- Worked through several Supabase RLS and session-token issues (inserts initially blocked by RLS until client was given the user's access token via `set_session()`).
- Also resolved a Streamlit CSS targeting issue where data-testid selectors didn't match across versions, requiring direct browser inspection to find the correct element to style.

- For now, the chat language is English – possible expansion to include other languages
- Next would be to guardrail this chat feature for prompt injection
- UI fixes are a major thing on the list – right now there is a basic UI with the dashboard page made cleaner
- No outstanding blockers currently

Day 4 — Optimizing Integration & Application Evaluation

Checklist:

- ☒ Evaluation approach defined (metrics + test set)
- ☒ Error handling + retries added (API) OR validation rules added (no-code)
- ☒ Prompt/model iteration based on failures
- ☒ Latency/cost notes captured
- ☒ Safety/guardrails considered (content, PII, injection)

Artifacts / notes:

Evaluation dataset link / location	test-set.md – 20 documented scenarios, 8 tested across 6 real uploads on one test account
Metrics used (accuracy, faithfulness, etc.)	Accuracy (correct numbers referenced In roast), crash-free rate (no errors on edge cases), tone consistency (Frank stays in character), actionability (ends with specific advice in every response)
Top failure modes found	1. Donut chart label overlap when category values are very small relative to dominant category 2. "vs Last Month" displaying misleading extreme percentages (e.g. +43495%) when consecutive months had drastically different totals 3. Frank occasionally acknowledged off-topic prompt injection attempts before redirecting
Fixes applied	1. Plotly uniformtext_mode='hide' to suppress overlapping labels 2. Percentage display capped at ±999% with "+" suffix for outlier swings 3. Topic-redirect guardrail added to Ask Frank prompt 4. Input validation added - income/budget capped at 1,000,000, net savings between -1,000,000 and 1,000,000 5. Duplicate month upload prevention - checks Supabase before inserting

	6. Basic logging added for Groq inputs/outputs and latency measurement
Latency/cost notes	Frank roast generates in ~1.85 seconds on llama-3.370b- versatile via Groq API. Ask Frank Q&A responses similar latency. No direct cost – Groq free tier used throughout development and testing

Decisions made today (why):

- Chose to cap percentage display rather than recalculate – math is correct, display was rather misleading; capping at +-999% preserves accuracy while preventing UX confusion
- Chose topic-redirect guardrail over hard content filtering – Frank’s character works better redirecting naturally than throwing an error
- Kept logging as print statements rather than a logging framework – sufficient for this hackathon scope – easy to upgrade later

Blockers / help needed:

- No outstanding blockers
- Session persistence on page refresh is known limitation – currently documented working on fixing it otherwise if not able to complete within given scope and timeline will be noted as a limitation
- Fix double sidebar, along with various card sizes, positioning of elements and colour of buttons
- From the Figma design – decision to be made on whether to include a budget vs actual progress bar (due to streamlit UI limitations)
- Similarly swap biggest category card for income card and have biggest category as a badge below the donut chart – matching the Figma outline
- Login & Sign-Up page needs to be cantered and put into a card right now it looks very basic – to be transformed into a much more appealing page

Summary of key Points for LLM/API Path: (obtained list from checkpoint for day 4)

1. Quality of API Responses:

Evaluated against accuracy, tone consistency, and actionability across 20 documented test scenarios - results in test-set.md.

2. Performance Metrics for LLMs:

BLEU scores not applicable for creative persona output - qualitative evaluation used instead, documented in test-set.md.

3. API Response Time:

Frank's roast averages 1.85 seconds via Groq; `st.spinner()` shown during all API calls, so users never see a blank screen.

4. Data Privacy:

No PII collected - only spending amounts and categories stored; Supabase RLS ensures users only access their own data; Groq receives spending summaries only.

5. Authentication:

All API keys stored in `.env` locally and Streamlit Secrets on deployment - never committed to GitHub, confirmed via `.gitignore`, excluded from logs.

6. User Interface Testing:

Full end-to-end flow tested manually - upload, dashboard, history, Ask Frank all verified working with clear error states and loading feedback throughout.

Day 5 — Integration of Model/API with Interface

Checklist:

- ☒ User flow designed (screens + inputs/outputs)
- ☒ Interface connected to backend/logic
- ☒ UX basics: loading, error states, reset/clear
- ☒ Logging of inputs/outputs enabled (safe logging)
- ☒ Demo link created (even if rough)

Artifacts / notes:

Interface tech (Gradio/Streamlit/React/No-code UI)	Streamlit
Demo link	To be deployed on day 6/7
Screenshots link	Refer to table below with screenshots for each page of the app – sample user logged in
Known UX issues to fix	Session lost on hard refresh Sidebar collapses and arrow disappears – workaround is page refresh with restores expanded state

Decisions made today

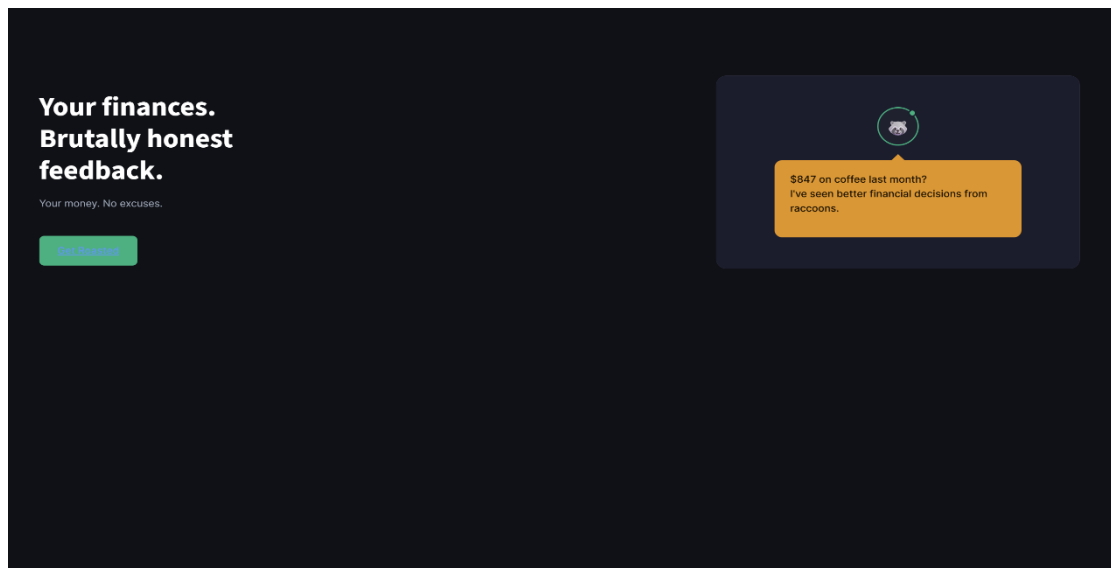
- Replaced full Ask Frank chat page with inline question box on the Dashboard page which reduces complexity while preserving the genuine AI interactivity
- The quick question feature allows one focused question which gets a better answer than open-ended chat anyway

- Frank's roast restructured into explicit 4-part format (opening observation, where it's going wrong, what's working, one specific recommendation) — initial single-paragraph output felt too generic; structured prompt produces significantly more specific and useful responses
- Budget vs Actual progress bar added with instant value feedback without mental calculation
- Loading spinners added on all Groq calls which prevents blank screen during AI generation
- Currency support dropdown added in the upload page selected currency is used across all data including history and dashboard and Frank's response
- Logo implemented in the sidebar as a wallet svg

Blockers / help needed:

- Testing the remaining scenarios and ensuring all or any bugs that come up are dealt with
- Deployment with smoke test and full functionality of app to be tested end to end
- Multi-currency use by the same user which would incorporate exchange rates to be implemented in future versions
- Full UI polish across the history page, upload page, login page and landing page
- Double sidebar to reverted to custom sidebar

Landing
Page



Login/Sign Up Page

Vault

Your money. No excuses.

Login

Sign Up

Email

your@email.com

Password

password

Login

Dashboard Page



Dashboard
Page



the GDP of a small island, and yet you're still managing to save some cash - I guess that's what I call a rent-controlled miracle. Your rent is literally half your budget, and I'm no economist, but I think that's a bit steep.

2. Where it's going wrong is that your 'food' and 'entertainment' categories are sneakily eating away at your budget, with AED 461 and AED 140 spent respectively - not staggering amounts, but when you're trying to save, every dirham counts, and it looks like you might be having a little too much fun.

3. What's actually working in your favor is that you're managing to save a decent chunk of change, AED 2,100 to be exact, which is no small feat considering your rent is through the roof - so kudos to you for being somewhat responsible with your cash.

4. Frank's advice is to cut your 'other' category spend by at least AED 100 and divert that to your savings, because let's be real, AED 160 on miscellaneous stuff is just a fancy way of saying you're throwing money out the window, and you could be throwing it into your savings account instead.

Ask Frank

Got a question about your spending?

e.g. Should I cut back on food?

Ask Frank

Upload
Page

app

Login

Dashboard

Upload

History

Vault

Dashboard

Upload

History

Frank

Still spending, I see.

Upload Statements

CSV or Excel files. Frank will do the rest.

"Go ahead, upload it. Let's see how bad the damage is this time around!"

Upload statement

Upload 10MB per file • CSV, XLSX

Select a month

June

Select Year

2026

Currency

\$

Income this month

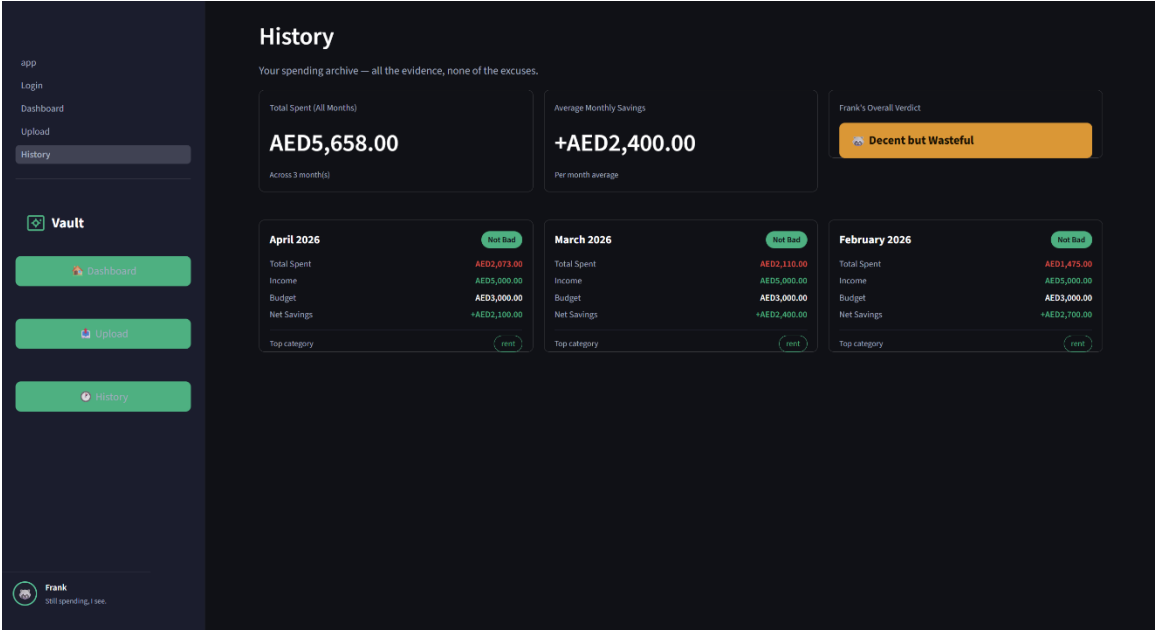
0

Budget this month

0

Net Savings

0

History Page	
-----------------	--

Day 6 — Final Enhancements, Security & Debugging

Checklist:

- ☒ Input validation + sanitization
- ☒ Secrets handling reviewed (no keys in repo)
- ☒ Rate limits / caching considered
- ☒ Bug list triaged and reduced
- ☒ README + architecture notes cleaned up

Artifacts / notes:

Security checklist notes	• API keys in Streamlit Secrets on deployment, .env locally, never committed. • Supabase RLS enforces row-level data isolation per user. • Groq calls log metadata only via print statements - user_id, latency, response length - no raw spending data. Input validation caps on all number fields (max 1,000,000). • File size capped at 10MB. Duplicate month uploads blocked at insert time.
Top bugs fixed	• Double sidebar showing Streamlit default nav - hidden via CSS targeting stSidebarNav.

	• Sidebar toggle arrow disappearing - fixed by setting <code>initial_sidebar_state="expanded"</code> on all pages. • Button text contrast - forced dark text globally via <code>.stButton > button</code> CSS. • Pillow incompatibility on Python 3.14 - removed pinned versions from requirements.txt. • Frank's verdict card overflowing container on History page - removed wrapper div. • Donut chart label overlap on small values - fixed with <code>Plotly uniformtext_mode="hide"</code>. • Extreme percentage display on vs Last Month - capped at $\pm 999\%$.
Remaining risks	• Session lost on hard refresh (Streamlit limitation). • Forgot password not implemented in v1. • Multi-sheet Excel reads first sheet only. • Groq rate limits not explicitly enforced — mitigated by design.

Decisions made today (why):

- Added the regenerate roast button, on the history page which when clicked on frank (calls Groq) which will generate a new roast
- Added a 30 second cooldown to the roast button so users don't accidentally spam click it and burn through groq tokens
- No rates or limit caching necessary since Frank generates his roast on current data and doesn't need to be recycled or reused as it has to match user's input data
- UI polish – history page: Resized the cards, to fit all the content for the summary of each month, adjusted Frank's verdict badge size
- UI polish – upload page: Rearranged drop downs to have month, year & currency in one row and income, budget and savings in another, implemented a preview of uploaded data so the user can see what they uploaded along with success and or error messages when file is uploaded, repositioned all elements to have a uniform grid
- UI polish landing page: added feature cards with details of the app, changed the call to action in the hero section to match the updated figma design
- UI polish – login page: implemented a typical card login/sign up with a simple toggle though the toggle may not look as professional due to streamlit limitations decided to go for a simple look
- UI polish overall app: removed double sidebar and changed the text colour across all buttons coloured green to black so as to make the text visible in comparison to the previous white colour against the green background
- Removed pinned package versions from requirements.txt – Streamlit clous uses Python 3.14 which was incompatible with pinned pillow==10.4.0

- Chose Streamlit Community Cloud over Render – free tier, direct GitHub integration, automatic redeployment on every push, no server configuration needed.

Blockers / help needed

1. Session is lost on refresh – this is a limitation
2. Forget password is not implemented in this version
3. Once deployed if user accidentally minimises sidebar they have to navigate to application tab in developer tools and clear all or open in incognito to see the sidebar again

Day 7 — Final Review & Deployment

Checklist:

- ☒ Deployment target chosen and deployed
- ☒ Environment variables set in deployment platform
- ☒ Smoke tests run on deployed version
- ☒ Performance checked (latency/cost)
- ☒ Submission package checklist started

Artifacts / notes:

Deployment platform	Streamlit
Deployment URL	https://vault-finance.streamlit.app/
Smoke test results	• Login and signup flow: PASS • File upload (CSV + XLSX): PASS • Supabase data save and fetch: PASS • Dashboard summary cards, charts, Frank's roast: PASS • History page month cards and verdict badges: PASS • Frank's overall verdict on History: PASS • Ask Frank question box: PASS • Multi-currency display: PASS • Session persistence within tab: PASS
Fallback plan if deployment breaks	• Redeploy from main branch on Streamlit Community Cloud. • Environment variables stored in Streamlit Secrets — no setup required beyond redeployment. • Local version runs via `streamlit run app.py` as backup demo option.

Decisions made today (why):

- Frank's Take colour change (dashboard + history + upload)
- Dashboard layout restructure (charts side by side, Frank below – ask and take)
- Month display line on dashboard to show current month data being displayed
- Quip rotation bottom corner of the sidebar
- Donut chart style update
- Frank's take updated to a more detailed and structured response format

Blockers / help needed:

- None

Day 8 — Final Submission & LinkedIn Sharing

Checklist:

- ☒ Final app link working
- ☒ Final repo is clean + documented
- ☒ Demo video recorded
- ☒ Final submission form completed
- ☒ LinkedIn post published + link shared

Artifacts / notes:

Final app URL	https://vault-finance.streamlit.app/
Final repo URL	https://github.com/bluejay-19/Vault
Demo video URL	https://drive.google.com/drive/folders/18mMUqSB1_plkDykzOr2WHizwc-hzGeZ?usp=sharing
LinkedIn post URL	https://www.linkedin.com/feed/update/urn:li:activity:7476509167361351680?utm_source=share&utm_medium=member_desktop&rcm=ACoAAFatjWkBDpTUZ6Q9oPQg6ZlCx-RLLGzqua4
Tweet/X URL (if applicable)	NA
Reflection: what you learned	Built a full-stack AI app from scratch in 8 days and learned more than any course could have taught me in the same time. Key lessons: Supabase RLS is powerful but will silently block everything until you pass the auth token correctly. Prompt engineering is less about clever tricks and more about structure - telling the model exactly what format you want produces dramatically better results than hoping it figures it out. CSS in Streamlit is a constant battle with internal component selectors that change between versions. The hardest part wasn't the AI integration - Groq is genuinely fast and easy to work with

	- it was the edge cases in data validation and making sure the app handles bad input gracefully without crashing. Frank ended up being genuinely funny because the prompt is grounded in real numbers rather than generic financial advice. Most importantly: shipping a working app beats having a perfect unfinished one.
--	---

Decisions made today (why):

- Chose not to implement forgot password in v1 - requires email confirmation to be re-enabled in Supabase and a separate reset flow; deferred to v2 to keep submission clean and on time
- Kept session persistence as a known limitation rather than implementing a fix - the browser localStorage approach would require significant refactoring right before submission; users can simply log back in

Blockers / help needed:

- None – the app is live; repo is clean & documentation is complete

Your finances. Brutally honest feedback.

Your money. No excuses.

Get Roasted

"You spent \$340 on miscellaneous last month. We both know what that means."



THIS MONTH

\$4,580 spent

Budget: \$3,500 - Over by \$1,080

\$0 131% of budget



Track Everything

Upload CSV or Excel statements. Frank catalogues every dollar you pretend you didn't spend.



Frank's Hot Takes

Unsolicited financial commentary from a raccoon who has seen your transaction history.



Private by Default

Your spending data stays yours. No ads, no selling your financial shame to third parties.



Vault

Your money. No excuses.

Login Sign Up

Email

your@email.com

Password



Login

Vault

Dashboard

Upload

History

Frank
Still spending, I see.

Dashboard

Your spending at a glance · August 2026

Total Spent

AED2,295.00

Income

AED3,000.00

vs Last Month

+91.2%

Net Savings to Date

AED12,000.00

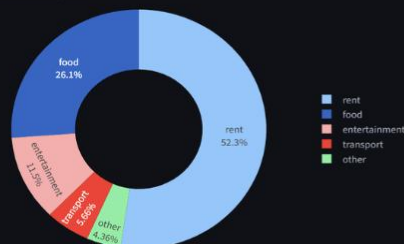
Budget vs Actual

AED0

AED2,295.00 / AED1,800.00 — AED495.00 over budget

AED1,800.00 budget

Where Your Money Went



Biggest category

Rent

Savings Trend



Vault

Dashboard

Upload

History

Frank

Still spending, I see.

Vault

Dashboard

Upload

History

Frank

The numbers don't lie.

Vault

Dashboard

Upload

History

Frank

Your wallet called, it's empty.

Frank's Take

OBSERVATION

You're shelling out a whopping AED 1200.00 for rent, which is basically the bulk of your expenses, and I'm starting to think you're secretly a landlord's best friend. That's more than half your budget, by the way, which is a pretty big chunk of change.

GOING WRONG

Your entertainment habits are also eating into your budget, with AED 265.00 spent on who-knows-what, probably binge-watching shows or eating out at fancy restaurants. And let's not forget the AED 130.00 you spent on transport, which I'm guessing is just you driving around in circles trying to find a parking spot.

WORKING

Honestly, not much is working in your favor this month, since you're actually in the red with a net savings of AED -200.00, but I guess it's good that you didn't spend anything on medical expenses or subscriptions.

ACTION

I'd recommend cutting back on your entertainment expenses by at least AED 100.00 next month, and putting that money towards getting your savings back on track, because clearly something's gotta give.

Ask Frank

Got a question about your spending? Frank won't sugarcoat it.

e.g. Should I cut back on food?

Ask Frank

Upload Statements

CSV or Excel files. Frank will do the rest.

"Go ahead, upload it. Let's see how bad the damage is this time around!"

Upload your monthly statement

Upload

10MB per file • CSV, XLSX

Month

June

Year

2026

Currency

\$

Income this month

0

Budget this month

0

Net Savings

0

History

Your spending archive — all the evidence, none of the excuses.

Total Spent (All Months)

AED14,953.00

Across 7 month(s)

Average Monthly Savings

+AED1,714.29

Per month average

Frank's Overall Verdict

Barely Scraping By

Regenerate Verdict

August 2026

Catastrophic

Total SpentAED2,295.00

IncomeAED5,000.00

BudgetAED1,800.00

Net SavingsAED-200.00

Top categoryrent

July 2026

Not Bad

Total SpentAED1,200.00

IncomeAED5,000.00

BudgetAED3,000.00

Net Savings+AED2,800.00

Top categoryfood

June 2026

Catastrophic

Total SpentAED3,795.00

IncomeAED5,000.00

BudgetAED3,000.00

Net SavingsAED-200.00

Top categoryrent

May 2026

Not Bad

Total SpentAED2,005.00

IncomeAED5,000.00

BudgetAED3,000.00

Net Savings+AED2,400.00

Top categoryrent

April 2026

Not Bad

Total SpentAED2,073.00

IncomeAED5,000.00

BudgetAED3,000.00

Net Savings+AED2,100.00

Top categoryrent

March 2026

Not Bad

Total SpentAED2,110.00

IncomeAED5,000.00

BudgetAED3,000.00

Net Savings+AED2,400.00

Top categoryrent

10) Community Sharing Tracker (bonus points)

Use this to track your social proof and claim points.

Date	Platform	Post URL	What you shared	Tagged DDS?	Points claimed
07/06/2026	LinkedIn	https://www.linkedin.com/feed/update/urn:li:activity:7469458711955009537?utm_source=share&utm_medium=member_desktop&rcm=ACoAAFaTjWkBDpTUZ6Q9oPQg6ZlCx-RLLGzqua4	Joined the DDS Build AI App Challenge – Group post	Yes	
07/06/2026	DDS Private Community	https://academy.decodingdataascience.com/c/#/post/13481	Joined the DDS Build AI App Challenge	Yes	
20/06/2026	Skool	https://www.skool.com/decoding-data-science-6929/day-1-of-the-8-day-ai-app-challenge?p=8976939f	Day 1- Defining the problem, target user and app concept	Yes	
20/06/2026	DDS Private Community	https://academy.decodingdataascience.com/c/#/post/13610	Day 1- Defining the problem, target user and app concept	Yes	
21/06/2026	DDS Private Community	https://academy.decodingdataascience.com/c/#/post/13633	Day 2 – Development environment setup and first working prototype	Yes	
21/06/2026	Skool	https://www.skool.com/decoding-data-science-6929/day-2-of-the-8-day-ai-app-challenge?p=7c5f7a50	Day 2 – Development environment setup and first working prototype	Yes	
22/06/2026	DDS Private Community	https://academy.decodingdataascience.com/c/#/post/13646	Day 3- Building the brain of the app	Yes	
22/06/2026	Skool	https://www.skool.com/decoding-data-science-6929/day-3-of-the-dds-ai-application-building-challenge?p=126fb512	Day 3 – Building the brain of the app	Yes	
23/06/2026	DDS Private Community	https://academy.decodingdataascience.com/c/#/post/13678	Day 4 – Optimizing API Integration & Application Evaluation	Yes	

23/06/2026	Skool	https://www.skool.com/decoding-data-science-6929/day-4-of-the-dds-ai-application-building-challenge?p=35a5dc0b	Day 4 – Optimizing API Integration and Application Evaluation	Yes	
24/06/2026	DDS Private Community	https://academy.decodingdata.science.com/c/#/post/13693	Day 5 – Integration of Model/API with interface	Yes	
24/06/2026	Skool	https://www.skool.com/decoding-data-science-6929/day-5-of-the-dds-ai-application-building-challenge?p=62fa9acf	Day 5 – Integration of Model/API with interface	Yes	
25/06/2026	DDS Private Community	https://academy.decodingdata.science.com/c/#/post/13708	Day 6 – Final Enhancements, Security and Debugging	Yes	
25/06/2026	Skool	https://www.skool.com/decoding-data-science-6929/day-6-of-the-dds-ai-application-building-challenge?p=23d93a28	Day 6 – Final Enhancements, Security and Debugging	Yes	
26/06/2026	DDS Private Community	https://academy.decodingdata.science.com/c/#/post/13722	Day 7 – Final Review and Deployment	Yes	
26/06/2026	Skool	https://www.skool.com/decoding-data-science-6929/day-7-of-the-dds-ai-application-building-challenge?p=90af7e2c	Day 7 – Final Review and Deployment	Yes	
27/06/2026	DDS Private Community	https://academy.decodingdata.science.com/c/#/post/13731	Day 8 – Final Submission	Yes	
27/06/2026	Skool	https://www.skool.com/decoding-data-science-6929/day-8-vault-is-submitted?p=0c4e55cd	Day 8 – Final Submission	Yes	
27/06/2026	LinkedIn	https://www.linkedin.com/feed/update/urn:li:activity:7476509167361351680?utm_source=share&utm_medium=member_desktop&rcm=ACoAAFaTjWkBDpTUZ6Q9oPQg6Zlclcx-RLLGzqua4	Day 8 – Final Submission	Yes	
26/06/2026	LinkedIn Comment	https://www.linkedin.com/posts/mdarshad_artificialintelligence-aichallenge-hackathon-share-7476220390848065537-	Presenting app to judges	Yes	

		1fQ8/?utm_source=share&utm_medium=member_android&rcm=ACoAAFatjWkBDpTUZ6Q9oPQg6Zlcx-RLLGzqua4			
09/06/2026	LinkedIn Comment	https://www.linkedin.com/posts/dds-business-circle_decodingdatascience-dds-aiapplicationchallenge-activity-7468607788068376577-Wcqb?utm_source=share&utm_medium=member_android&rcm=ACoAAFatjWkBDpTUZ6Q9oPQg6Zlcx-RLLGzqua4	Joining the challenge	Yes	

11) Reference Resources (from the challenge)

Core challenge walkthrough video: <https://www.youtube.com/watch?v=X4PitcxNDjE>

Python environment setup: <https://www.youtube.com/watch?v=D5XyQ96EgiM&t=759s>

GitHub basics: https://youtu.be/bV_9mr5O5bg?si=0oiiT9BZutcKCjnL

OpenAI API key setup: <https://www.youtube.com/watch?v=CVnTzj-qhCU&t=8s>

Model/LLM selection blogs:

- <https://decodingdatascience.com/choosing-the-right-gemini-model-for-your-ai-project-a-beginner-friendly-guide/>
- <https://decodingdatascience.com/openai-guide-to-use-which-model-for-tasks/>
- <https://decodingdatascience.com/how-to-choose-the-right-openai-model-gpt-5-complete-guide/>

AI Explorer RSVP: <https://nas.io/artificialintelligence/events/ai-explorer-ai-demos-ai-use-cases-and-q-a-1767887350300>

12) Reviewer / Mentor Notes (optional)

Use this section if an instructor/mentor is reviewing your progress.

Strengths observed:

-

Areas to improve next:

-

Action items:

-